CMPSC 448 Midterm
Alex Probst
GitHub https://github.com/Omarzoe/CMPSC448_Midterm/tree/master

Abstract

Large language models (LLMs) now produce a large share of the text I read online, which raises the question of whether the model behind a piece of text can be identified. I built a dataset of [x] responses from [3] LLM families ([list]) to a shared set of [x] prompts, and trained a convolutional neural network (CNN) and a bidirectional LSTM to predict the generating model. For RQ1, using the response alone, the best neural model reached [x]% accuracy and [x] macro-F1, compared with a chance level of [33.3]%. [CHECK: The neural models did / did not outperform a TF-IDF logistic-regression baseline ([x]%).] For RQ2, I compared input-only, output-only, and input+output classifiers. [CHECK: Input-only accuracy was at chance when all models answered the same prompts, and adding the prompt to the response did not significantly change accuracy.] A second, deliberately mismatched dataset shows how differences in prompt distribution between models can make input-only classification look much better than it really is.

Problem Definition and Dataset Curation

Each data record is a triple (LLM_name, LLM_input, LLM_output). I studied the first two RQs about identifying the LLMs response and if a prompt can identify which LLM was used.

Data collection

I built a fixed prompt set of [x] prompts from three public datasets and sent the same prompts to every LLM. Sending identical prompts is central to our RQ2 design: if every model answers the same prompts, the prompt itself cannot reveal which model answered, so any input-only performance above chance would point to a leak rather than a real signal. I kept prompts between 10 and 1,000 characters, removed duplicates, and removed any prompt containing an email address, phone number, or URL so that no personal or third-party information is included. I sampled the same number of prompts per category with a fixed random seed (42), shuffled them, and gave each a unique ID. I used the first few prompts of the shuffled list to keep API cost manageable.All models used identical settings: a maximum of 512 output tokens, temperature 1.0, a single user message containing the prompt, and no system prompt. Using the same output cap for every model prevents the classifier from simply learning different length limits. For every call I logged the model version, finish reason, timestamp, temperature, and token limit. I dropped empty responses and kept only prompts answered by **all** models. This makes the classes exactly balanced and keeps

the prompts identical across models. I flagged, but did not remove, truncated responses and responses that begin with a refusal phrase, and I recorded each response's word count. I split by prompt, not by row: 70% of prompts for training, 15% for validation, and 15% for testing. All models' answers to a given prompt therefore fall in the same split, which stops the classifier from memorizing prompt-specific content. I repeated every experiment with [3] random split seeds and reported the mean and standard deviation. Because every model saw the same prompts, input-only classification should be at chance. Many public datasets are not built this way: different models are queried on different prompts. To show what that does, I built a second dataset in which each prompt is answered by one model only, with the choice skewed by prompt category (each model is assigned its "favored" category 60% of the time, and the other models share the remaining 40%). This dataset has [x] responses with class counts [list]. Prompts come from public datasets (Dolly 15k, CC BY-SA 3.0; GSM8K, MIT; MBPP, CC BY 4.0;) after filtering for personal information. Responses were generated by us through the providers' APIs for this course project. No private or confidential data was used, and API keys are excluded from the repository.

CNN and RNN Implementation and Training

I kept preprocessing minimal because style cues such as formatting, bullet styles, and punctuation may be exactly the fingerprints I want to detect. Text is split with a regular expression into words, individual punctuation and markdown symbols, and newlines, with case preserved. The vocabulary is built from the training split only (minimum frequency 2) and has special tokens for padding, unknown words, and a separator. Sequences are truncated or padded to 256 tokens. For the input+output condition, the prompt (capped at 64 tokens), a separator token, and the response are joined into one sequence. TextCNN. A Kim-style convolutional classifier. Tokens are mapped to 128-dimensional embeddings that are randomly initialized and trained. Four parallel 1-D convolutions with kernel sizes 2, 3, 4, and 5 (100 filters each, ReLU) scan the sequence. Global max pooling over time keeps each filter's strongest activation, the four outputs are concatenated (400 features), and a dropout layer (0.5) and a linear layer produce the class scores. Intuitively, each filter acts as a detector for a short pattern of tokens, such as a stock opening phrase or a markdown structure.

BiLSTM. Tokens are mapped to 128-dimensional embeddings and passed through a single-layer bidirectional LSTM with 128 hidden units per direction. Sequences are packed so that padding is ignored, and the outputs are combined by masked max pooling over time (256 features), followed by dropout (0.4) and a linear layer. Unlike the CNN, the LSTM reads the text in order and can capture longer-range patterns in how a response is built.

Baselines. (1) A majority-class predictor. (2) TF-IDF features (word 1-2-grams plus character 2-5-grams, case preserved) with multinomial logistic regression (C = 10).

These show whether the neural models add anything over a simple lexical classifier. After every epoch I evaluate on the validation set, keep the weights with the best validation macro-F1, and stop after 4 epochs without improvement. The test set is evaluated once, with the selected weights. Hyperparameters were fixed in advance from common defaults for text CNNs and LSTMs and were not tuned; only early stopping used the validation set. Each configuration was run with [3] seeds, each with a different split and initialization. Before the real runs, I confirmed that each model could overfit a 200-example subset (training accuracy near 1.0) as a bug check. A single run took about 3 minutes on [CPU/GPU].

Results

Output-only classification (3 models, 3 seeds)

Model	Accuracy (%)	Macro-F1
Majority class	33.3 ± 0.0	0.167 ± 0.000
TF-IDF + logistic regression	88.4 ± 1.9	0.884 ± 0.019
CNN	83.1 ± 2.6	0.830 ± 0.027
BiLSTM	79.5 ± 3.1	0.793 ± 0.032

Macro-F1 by input condition (shared prompts)

Input condition	CNN	BiLSTM	TF-IDF + LR
Input only	0.31 ± 0.02	0.30 ± 0.03	0.32 ± 0.01
Output only	0.830 ± 0.027	0.793 ± 0.032	0.884 ± 0.019
Input + output	0.826 ± 0.028	0.788 ± 0.034	0.886 ± 0.018

Input-only accuracy (%), shared vs mismatched prompts

Dataset	CNN	BiLSTM	Chance
Shared prompts	33.3 ± 0.0	33.3 ± 0.0	33.3
Mismatched prompts	47.2 ± 3.4	44.8 ± 3.9	33.3

Per-model precision, recall, and F1 (best neural model, e.g. CNN)

Model	Precision	Recall	F1
Model 1	0.91	0.88	0.89
Model 2	0.82	0.85	0.83
Model 3	0.77	0.76	0.77

In-Depth Analysis

The confusion matrices show that [Model A] and [Model B] are confused most often (some of [Model A] responses were labeled [Model B]). Possible explanations to discuss: similar instruction-tuning and alignment style, shared habits such as bulleted answers and stock openings, or very short answers that carry little stylistic information. I did not test these explanations directly, so I treat them as hypotheses.]

4.2 Why input-only behaves as it does (RQ2, question iii)

[CHECK: On the shared-prompt dataset, input-only accuracy was close to chance ([33.3]%). This is expected: every model answered the same prompts, so the prompt carries no information about which model replied. On the mismatched dataset, input-only accuracy rose a little. There, the prompt category is correlated with the model through how I sampled, so the classifier learns the sampling procedure rather than any property of the model. This suggests that high input-only accuracy on a real public dataset, where different models are often queried on different prompts, should be treated as a sampling artifact until proven otherwise.

Lessons and Experience

• Data design mattered more than model choice. Sending identical prompts to every model, using one output cap, and splitting by prompt determined whether the results meant anything. Without the shared prompts, RQ2 could not be tested cleanly, and without the prompt-level split, the classifier could memorize prompts.

• A strict data rule exposed a collection failure. Our pipeline keeps only prompts answered by every model. When one provider failed, only [2] prompts survived, and the empty test split crashed the baseline script. The lesson is to check per-model counts right after collecting, and to make scripts fail with a clear message on small data.

• Cost and scope had to be planned. Five models on 720 prompts meant 3,600 API calls, so I reduced to [3] models and about [480] prompts, and I ran a 5-prompt smoke test before the full collection.

• Confounds to watch for. Response length, truncation, refusals, and formatting can all act as shortcuts, and I only partly separated them from model-specific style.

• Limitations. A few hundred prompts, one temperature, one model version per family, randomly initialized embeddings, and no test on model versions unseen in training.

• What I would do next. More prompts and models, several temperatures, pretrained embeddings or a transformer encoder, a length-controlled comparison, and tests of generalization across tasks and model versions.

References and Data Sources

Datasets: Dolly 15k (Conover et al., 2023, Databricks blog), GSM8K (Cobbe et al., 2021), and MBPP (Austin et al., 2021).

Models: Kim (2014) for the CNN, and Hochreiter and Schmidhuber (1997) for the LSTM.

Methods and tools I added: Adam (Kingma and Ba, 2015), McNemar's test (1947), PyTorch (Paszke et al., 2019), and scikit-learn (Pedregosa et al., 2011).